

Research Review of Your NexusForge Prompt and Streamlit Implementation Path

Bottom line

The files you uploaded are not a finished codebase. They are specification documents: the original file is a “Master Development Prompt” for a large social media automation and AI content platform, and the enhanced NexusForge files are expanded product requirements for an enterprise SaaS with dashboards, CRM, agency management, AI integrations, billing, and settings. They describe what to build, but they do not provide a runnable Python + Streamlit implementation that can be reviewed, debugged, or completed as existing source code. [filecite:turn0file4](#) [filecite:turn0file2](#) [filecite:turn0file3](#)

That means the right conclusion is not “the code is almost done.” The right conclusion is “the current materials are a broad PRD and prompt pack, but they still need to be translated into a scoped implementation prompt for a Python-only Streamlit system.” Your current documents also target React/TypeScript/Next.js and Node/FastAPI patterns, which is materially different from your latest requirement of “Python and Streamlit code only.” [filecite:turn0file4](#) [filecite:turn0file2](#)

What your uploaded files actually specify

The original prompt file defines a production-grade platform using React 18+, TypeScript, TailwindCSS, Framer Motion, PostgreSQL, Redis, Node/Express or Python/FastAPI, Bull/Redis or Celery, and multiple AI providers. It lays out a very large surface area: authentication, dashboard widgets, multi-account management, scraping, scheduling, CMS, social publishing, email, analytics, SEO, and billing. That is a product blueprint, not an implementation repository. [filecite:turn0file4](#)

The enhanced NexusForge prompt goes even further. It specifies Next.js 15, App Router, Tailwind v4, shadcn/ui, TipTap, Recharts, Zustand, React Query, Node.js 20+, Python 3.12, PostgreSQL, Redis, Meilisearch, ClickHouse, Stripe, agency/client management, CRM, sales pipeline, AI Studio, reports, platform settings, and an AI provider matrix that includes NVIDIA NIM, Anthropic, Google, xAI, and OpenAI. It also explicitly defines platform settings for AI model API keys, social platform API credentials, and email provider settings. Again, this is detailed and useful, but it is still a requirements document, not deployable Streamlit code. [filecite:turn0file2](#) [filecite:turn0file3](#)

A second important finding is alignment: your enhanced prompt is optimized for a custom web application stack with a sophisticated front end, while your latest instruction asks for a single Python + Streamlit implementation path. Those are not equivalent delivery modes. The former assumes a dedicated front-end architecture; the latter is best treated as a corporate control plane or internal SaaS app written entirely in Python. [filecite:turn0file2](#) [filecite:turn0file4](#)

What Streamlit can realistically support for this project

Streamlit is now capable enough to support a serious Python-first internal application. Its official docs recommend `st.navigation` as the preferred and more customizable way to build multipage apps, with the entrypoint file acting as the router or shared frame around pages. Streamlit also supports grouped navigation sections, shared widgets, and per-session state across a multipage app. ¹

Streamlit also has first-party secrets management. Secrets can live in project-level or global `.streamlit/secrets.toml` files, should be kept out of Git, and can be accessed both through `st.secrets` and as environment variables. That makes it a good fit for securely wiring OpenAI, Claude, Gemini, xAI, NVIDIA NIM, database credentials, and application secrets into a Python-only app. ²

For login, Streamlit now supports built-in OIDC authentication. The official docs document `st.login()`, `st.user`, and `st.logout()`, and explicitly call out Google Identity and Microsoft Entra ID as supported identity-provider patterns. That is directly relevant to your request for a corporate admin login experience. ³

The main limitation is just as important: Streamlit's built-in OIDC support is for authentication, not authorization. The docs explicitly say OIDC tells you who the user is, but does not give your app authority to impersonate them, and they recommend a custom component if you need generic OAuth 2.0 or app-to-provider actions on behalf of the user. That matters because your prompt includes many "connect platform account and act for the user" flows. In a Streamlit build, the safest path is to use Streamlit OIDC for app login and then use provider-specific API tokens or carefully designed OAuth helper flows for external platforms, rather than storing raw third-party usernames and passwords. ⁴

There are also flow-design constraints that matter for your spec. Streamlit forms must include `st.form_submit_button`, and inside a form, that submit button is the only widget that can have a callback. Streamlit session state is also tied to the user session and can reset on browser reload or direct URL navigation. In practice, that means large enterprise onboarding wizards, content planners, and approval flows should be built around explicit submit steps and database persistence, not around fragile in-memory UI state. It also means durable automation jobs should not depend on the UI session itself. ⁵

What current official API docs support

Your multi-provider AI direction is technically sound, but the integration layer needs to be normalized carefully. OpenAI's current quickstart shows the Python SDK using the Responses API and an `OPENAI_API_KEY` environment variable. Anthropic's current API overview documents a REST API at `https://api.anthropic.com`, with the direct API requiring either `x-api-key` or `Authorization`, plus `anthropic-version`. Google's Gemini docs show both the Python `google.generativeai` client and REST calls using `x-goog-api-key`. xAI's current docs show that Grok can be called through xAI's own SDK or through the OpenAI client pattern by setting `base_url="https://api.x.ai/v1"`. ⁶

That combination is useful architecturally. It means a Python provider-abstraction layer is realistic: one adapter for OpenAI, one for Anthropic, one for Gemini, one for xAI, and one for NVIDIA NIM, all behind a common service interface. In Streamlit, those provider keys should live in `st.secrets` or environment variables, not in hard-coded Python files and not in plain text committed to source control. ⁷

Your uploaded NexusForge spec already points in this direction conceptually by defining AI-provider settings, model routing, provider test connections, and a model matrix that assigns different use cases to different vendors. The missing piece is not product thinking; it is implementation scoping and a Python-native service design. turn0file2turn0file3

What needs correction in your NVIDIA NIM plan

The strongest technical correction from the research is that your pasted NVIDIA section is too simplified. The official NVIDIA NIM docs do support OpenAI-compatible chat completions at `https://integrate.api.nvidia.com/v1/chat/completions`, and those endpoint docs explicitly show Bearer credentials and OpenAI-style chat request structure.  8

However, the official docs do **not** show one universal base URL for every capability. The embedding endpoint documentation uses `https://ai.api.nvidia.com/v1/retrieval/nvidia/embeddings`. The reranking docs use `https://ai.api.nvidia.com/v1/retrieval/nvidia/llama-3_2-nemoretriever-500m-rerank-v2/reranking`. The image-generation docs for Flux use `https://ai.api.nvidia.com/v1/genai/black-forest-labs/flux.1-dev`. All of those also use Bearer credentials, but they are a different endpoint family from the OpenAI-compatible chat route. So the statement “all endpoints use the same base URL” is not accurate enough for implementation. A capability-aware NVIDIA adapter is the correct design.  9

The current NVIDIA catalog is also dynamic enough that hard-coding a fixed “77 free models” count is risky. The public docs page currently lists models including `z-ai / glm5.1`, `qwen / qwen3-5-122b-a10b`, `moonshotai / kimi-k2-instruct`, `moonshotai / kimi-k2-thinking`, and also newer entries such as `moonshotai / kimi-k2.6`. The build portal itself advertises “free inference with leading models,” but it does not expose a stable static count in the page snippet I could verify. The safer implementation choice is to treat the docs and portal as the source of truth and avoid freezing a numeric model count into code or prompt text.  10

In other words, NVIDIA NIM belongs in your system, but it should be integrated as a routed provider with separate handling for chat, retrieval, rerank, and image endpoints, not as a single blanket “base URL + 77 models” block.  11

What should be removed or redesigned before code generation

Your current prompt pack includes a large number of features that are either unsupported by the official integration patterns I verified, over-scoped for Streamlit, or inappropriate to turn into an implementation brief: raw social login credential capture, login-challenge solving, CAPTCHA solving, proxy rotation, auto-follow/unfollow, auto-like, auto-comment, auto-DM, story-view bots, and mass scraping of competitor followers and likers. Those items appear throughout the uploaded documents, but they do not line up with the official, documented API-key/OIDC/token patterns shown in the current provider docs I reviewed.

turn0file4turn0file2  12

For a real Python + Streamlit build, those should be replaced with a compliant operations stack: authorized website/CMS connectors, content ingestion from your own properties, AI drafting, editorial approval, official social publishing, analytics, moderation where supported, lead capture, CRM sync, reporting, and provider

orchestration. That is still a powerful “AI automation revenue engine,” but it is grounded in supportable integrations and far more likely to become a working system instead of a brittle or unsafe one.

filecite turn0file2 filecite turn0file3 13

A second redesign point is execution model. Because Streamlit reruns the script and ties session state to the current session/WebSocket, long-running or scheduled automations should live in separate Python service modules or scheduled workers, even if the entire project remains Python-only. Streamlit should be the control plane and operator console; durable jobs should persist state in a database and run independently from the browser session. That is an inference from Streamlit’s documented navigation, rerun, and session-state behavior, and it is the right way to make a 100-project corporate system reliable.

14

Single consolidated master prompt

The prompt below is the research-corrected version I would use. It is aligned to your “Python + Streamlit only” requirement, it preserves the strongest business parts of your NexusForge concept, and it removes the parts that should not be turned into implementation code. It is also shaped around the current official documentation patterns for Streamlit auth/secrets and the major AI providers above. 15

filecite turn0file2

You are a principal Python architect, senior Streamlit engineer, and AI-platform builder.

Build a production-ready, corporate-looking, multi-workspace application called “NexusForge AI Revenue Engine” using Python and Streamlit as the primary stack. Generate runnable, well-structured code only. Do not return essays, plans, or pseudo-code. Return a complete file tree and the full contents of every file needed to run the system.

PRIMARY GOAL

Create a secure enterprise operations platform for a Sales and Marketing Director managing 100+ projects. The app must unify:

- website and CMS connection for the user’s own properties
- AI content generation across multiple providers
- official social publishing and analytics workflows
- CRM and lead pipeline tracking
- executive dashboard and revenue reporting
- centralized settings and provider-key management
- audit logging and easy configuration

IMPORTANT BOUNDARIES

Do NOT implement:

- raw social-media password capture
- CAPTCHA solving

- login-challenge solving
- proxy rotation and stealth/bypass features
- auto-follow, auto-unfollow, auto-like, auto-comment, auto-DM, story-view bots
- scraping of private or unauthorized account data
- anything that impersonates users or violates official platform access patterns

Instead:

- use official APIs, official tokens, or approved OAuth/token-based connectors
- accept website URL plus authorized connector credentials for the user's own systems only
- store secrets securely
- build compliant publishing, analytics, moderation, content, CRM, and reporting workflows

TECH STACK

Use:

- Python 3.12+
- Streamlit for UI
- st.navigation for multipage routing
- st.session_state only for session UI state, not for durable business state
- SQLAlchemy ORM
- SQLite by default, but structure the code to support PostgreSQL through environment configuration
- Pydantic for settings and validation
- requests or httpx for API calls
- pandas for tables and exports
- plotly for charts
- passlib or bcrypt for local admin password hashing if local login is used
- Streamlit built-in OIDC support for Google/Microsoft app login if configured
- service layer + repository layer + adapters pattern
- background job module in Python for scheduled publishing/report generation, separated from the main Streamlit UI session

PROJECT ARCHITECTURE

Generate a clean project like this:

- app.py
- core/
 - config.py
 - auth.py
 - security.py
 - database.py
 - logging.py
 - navigation.py
- models/
 - user.py
 - workspace.py

- provider.py
- website.py
- content.py
- social.py
- crm.py
- analytics.py
- audit.py
- repositories/
- services/
 - ai_router.py
 - providers/
 - openai_provider.py
 - anthropic_provider.py
 - gemini_provider.py
 - xai_provider.py
 - nvidia_nim_provider.py
 - website_service.py
 - content_service.py
 - social_service.py
 - crm_service.py
 - analytics_service.py
 - report_service.py
 - scheduler_service.py
- pages/
 - dashboard.py
 - workspaces.py
 - website_hub.py
 - ai_studio.py
 - content_planner.py
 - social_publisher.py
 - crm_pipeline.py
 - analytics_reports.py
 - settings.py
 - audit_log.py
- components/
 - kpi_cards.py
 - charts.py
 - tables.py
 - forms.py
 - status_badges.py
- jobs/
 - publish_jobs.py
 - report_jobs.py
- data/
 - seed.py
- .streamlit/
 - config.toml
 - secrets.toml.example

- requirements.txt
- README.md

USER EXPERIENCE AND DESIGN

Make the interface fully corporate, polished, and easy to use:

- dark enterprise default theme
- clear sidebar navigation with grouped sections
- KPI cards, filters, charts, and executive summaries
- compact, professional spacing
- forms that are explicit, stable, and easy to configure
- no broken placeholders
- real demo data for 100+ projects, clients, campaigns, and leads
- searchable tables and CSV export
- settings-first configuration so a non-developer can operate the app

AUTHENTICATION

Support two app-login modes:

- Streamlit OIDC mode for Google or Microsoft when configured
- local admin mode for development/demo with hashed passwords

Create role support:

- Super Admin
- Admin
- Director
- Manager
- Analyst
- Viewer

Add workspace isolation:

- multiple clients/workspaces
- per-workspace website connections
- per-workspace AI provider defaults
- per-workspace social connections
- per-workspace CRM pipeline
- per-workspace audit logs

WEBSITE HUB

Build a "Website Hub" page that accepts:

- website URL
- company name
- brand description
- optional authorized CMS/API connector credentials for the user's own site
- optional sitemap URL or RSS feed
- optional webhook endpoints

Website Hub features:

- crawl/import public pages from the provided site URL
- parse titles, headings, summaries, and primary topics
- generate AI summaries of website positioning
- create content briefs from website pages
- build draft blog posts and social snippets from site content
- track connected websites per workspace
- show site pages imported, last sync, and sync status
- include read-only mode if no authenticated CMS connector is available

Do not require raw website admin passwords unless the user is connecting their own system through an officially supported app-password or token method. Store only secure connector data.

AI STUDIO

Build a multi-provider AI hub supporting:

- OpenAI
- Anthropic
- Google Gemini
- xAI Grok
- NVIDIA NIM

Requirements:

- provider settings page for API keys, default models, enable/disable toggles, and test connection
- centralized model router
- per-task provider selection
- fallback strategy if one provider fails
- token/cost tracking fields
- prompt templates library
- output history
- save generated output to drafts

Use cases:

- blog post generation
- social caption generation
- website summary generation
- landing-page rewrite
- SEO metadata generation
- email copy generation
- sales outreach draft generation
- executive summary generation

NVIDIA NIM adapter requirements:

- do not assume one universal endpoint for every capability
- support chat-style inference through a chat adapter
- support capability-aware routing for retrieval/embedding/rerank/image modules

- make the adapter extensible and model-driven
- include model registry configuration rather than hard-coding a fixed static list count

SOCIAL PUBLISHER

Build a compliant social publishing console for officially connected accounts only.

Support these managed concepts:

- connected platform accounts
- token status
- publish capability status
- account nickname
- workspace assignment

Focus on allowed workflows:

- create post
- schedule post
- platform-specific caption variations
- content calendar
- approval status
- publishing history
- performance snapshots
- comment/inbox moderation placeholders only where officially supported
- analytics import from official APIs or CSVs

Do NOT implement any behavior automation around following, liking, mass messaging, stealth, scraping, or account bypass.

CONTENT PLANNER

Create:

- editorial calendar
- drafts
- scheduled content
- published content
- content tags
- campaign labels
- approval workflow
- duplication/repurpose tools
- one-click generate social variants from blog/article drafts
- one-click generate blog/article drafts from website pages or campaign briefs

Add content templates for:

- blog articles
- LinkedIn posts
- Instagram captions

- Facebook posts
- Pinterest descriptions
- sales email drafts
- landing-page copy
- monthly reports

CRM AND REVENUE ENGINE

Build a practical CRM page with:

- contacts
- companies
- source channel
- stage
- value
- owner
- follow-up date
- notes
- activity history

Pipeline stages:

- New
- Qualified
- Meeting
- Proposal
- Negotiation
- Won
- Lost

Add:

- Kanban board
- table view
- contact detail drawer
- revenue totals
- forecast totals
- lead-source summary
- simple attribution fields connecting content/social/website activity to leads and revenue
- CSV import/export

EXECUTIVE DASHBOARD

Create a polished dashboard showing:

- active workspaces
- connected websites
- connected provider status
- content generated this week
- content scheduled
- leads created

- pipeline value
- won revenue
- tasks due
- provider usage summary
- recent audit events

Include charts for:

- content output over time
- leads by source
- revenue by workspace
- pipeline by stage
- provider usage by task type

ANALYTICS AND REPORTS

Build reporting pages for:

- workspace summary
- website content summary
- social publishing summary
- CRM funnel summary
- revenue summary
- provider usage summary

Add:

- date filters
- workspace filters
- export to CSV
- export to PDF-ready HTML or styled markdown snapshot
- executive report generator using AI

SETTINGS

Create a strong Settings page with tabs for:

- General
- Authentication
- Workspaces
- AI Providers
- Social Connections
- Website Connectors
- CRM Defaults
- Branding
- Notifications
- Audit and Logs

Use secure configuration patterns:

- read secrets from Streamlit secrets or environment variables
- never hard-code credentials
- provide a secrets example file

- give clear validation errors if required keys are missing

DATABASE AND DEMO DATA

Seed realistic demo data:

- 100+ projects/workspaces
- multiple users and roles
- website records
- AI provider configs
- content drafts and scheduled items
- CRM contacts and deals
- analytics samples
- audit events

QUALITY BAR

The output must:

- run locally with a clear README
- install from requirements.txt
- include database initialization
- include seed script
- avoid placeholder-only pages
- keep code modular and readable
- include comments only where useful
- use type hints
- have basic error handling
- validate settings on startup
- fail gracefully when provider keys are absent

OUTPUT FORMAT

Return:

1. full file tree
2. complete code for every file
3. requirements.txt
4. .streamlit/config.toml
5. .streamlit/secrets.toml.example
6. minimal README with exact run steps

Do not return explanations outside the code deliverable.

This is the version of your prompt that is strongest from both a product and implementation perspective: it preserves the enterprise sales/marketing value of the NexusForge concept, fits a Python + Streamlit build, supports OpenAI/Claude/Gemini/Grok/NVIDIA NIM cleanly, and avoids the parts of the earlier documents that should not be turned into working code. [filecite turn0file2](#) [filecite turn0file4](#) 16

1 <https://docs.streamlit.io/develop/concepts/multipage-apps/overview>

<https://docs.streamlit.io/develop/concepts/multipage-apps/overview>

2 7 15 16 <https://docs.streamlit.io/develop/concepts/connections/secrets-management>

<https://docs.streamlit.io/develop/concepts/connections/secrets-management>

3 4 12 13 <https://docs.streamlit.io/develop/concepts/connections/authentication>

<https://docs.streamlit.io/develop/concepts/connections/authentication>

5 <https://docs.streamlit.io/develop/api-reference/execution-flow/st.form>

<https://docs.streamlit.io/develop/api-reference/execution-flow/st.form>

6 <https://platform.openai.com/docs/quickstart>

<https://platform.openai.com/docs/quickstart>

8 11 https://docs.api.nvidia.com/nim/reference/abacusai-dracarys-llama-3_1-70b-instruct-infer

https://docs.api.nvidia.com/nim/reference/abacusai-dracarys-llama-3_1-70b-instruct-infer

9 <https://docs.api.nvidia.com/nim/reference/nvidia-embedding-2b-infer>

<https://docs.api.nvidia.com/nim/reference/nvidia-embedding-2b-infer>

10 <https://docs.api.nvidia.com/nim/reference/models-1>

<https://docs.api.nvidia.com/nim/reference/models-1>

14 https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state

https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state